HPC & Parallel Programming

MPI Project Lab

Kun Suo

Computer Science, Kennesaw State University

https://kevinsuo.github.io/

Part 1: matrix multiplication

$$C = AB$$

$$C_{m,n} = \sum_{n=1}^{N} A_{m,n} B_{n,k}$$

Single thread solution

```
void MultiplyMatrices(int nCount, double **matrixA,
                      double **matrixB, double **matrixC)
{
    int i, j, k ;

    for (i = 0; i < nCount; i++)
    {
        for (j = 0; j < nCount; j++)
        {
            matrixC[i][j]=0;

            for (k = 0; k < nCount; k++)
            {
                matrixC[i][j] +=
                    matrixA[i][k]*matrixB[k][j];
            }
        }
    }
}
```

Part 1: matrix multiplication

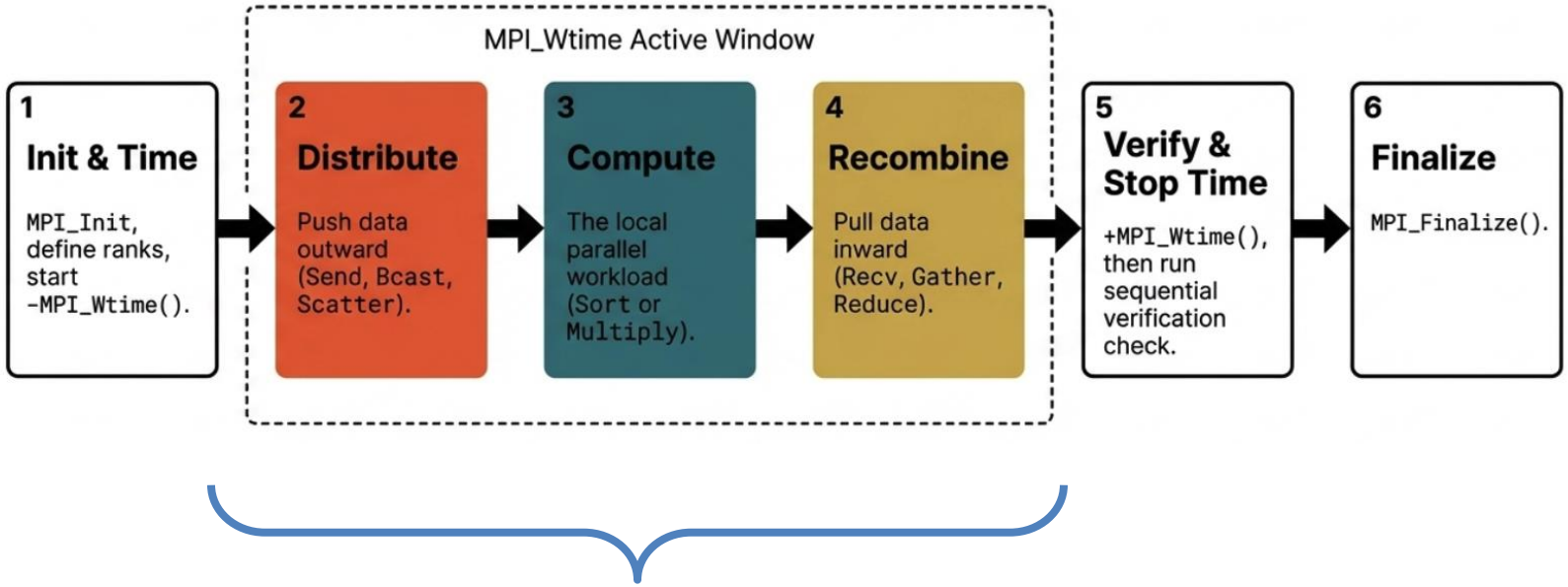
- Problem: Implement a parallel solution using MPI for general matrix multiplication
  - Input: Three integers M, N, and K (512 ~ 2048)
  - Problem Description: Randomly generate two matrices A (M*N) and B (N*K), and perform matrix multiplication to obtain matrix C.
  - Output: The time taken for the matrix calculation.

Part 1: matrix multiplication

- In your implementation, you must include a verification function that checks whether the output matrix C produced by the MPI optimization is identical to the original matrix C. (Note that if your matrix elements are floating-point numbers, elements within a certain tolerance can be considered the same. You can set a very small threshold (Epsilon), such as 10^{-9}. The equality check formula is: $|a - b| < \varepsilon$. If the difference between a and b is less than this small value, we consider them "equal".) The time of verification function should not be included in the parallel solution.

Part 1: matrix multiplication

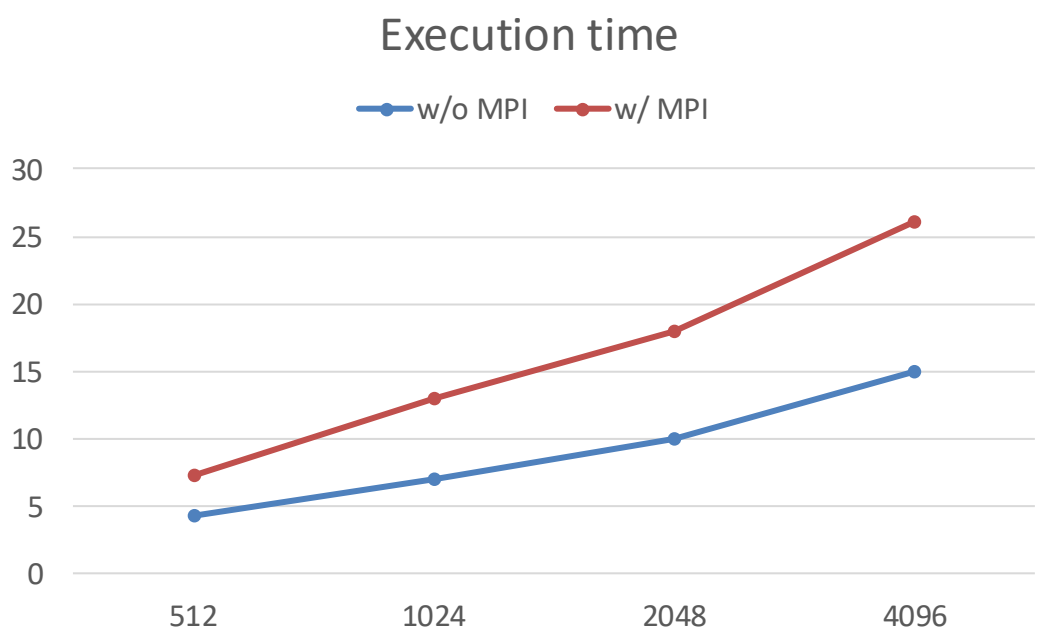
- The Anatomy of a Parallel Algorithm



Parallelization & Speedup

Part 1: matrix multiplication

- Defining Success Through Performance
  - Parallel using MPI should speedup against sequential execution.

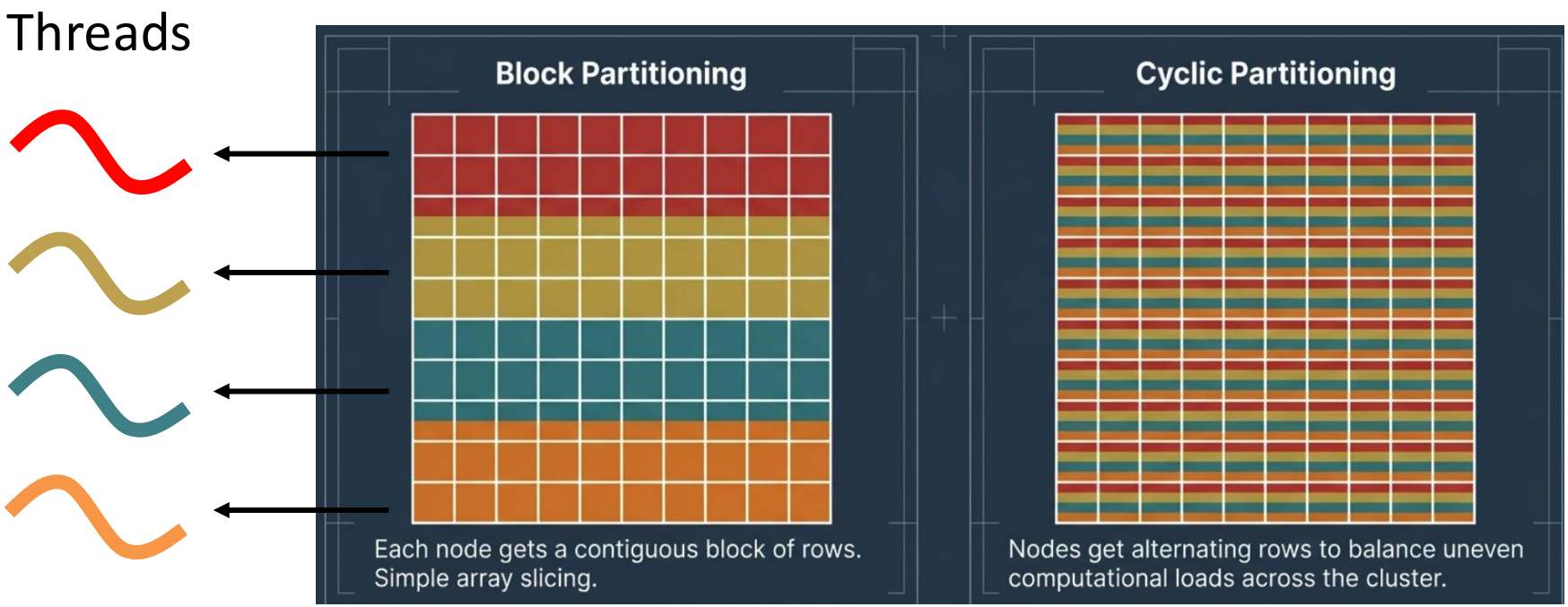


Speedup

$$S(n,p) = \frac{T_{\text{serial}}(n)}{T_{\text{parallel}}(n,p)}$$

Part 1: matrix multiplication

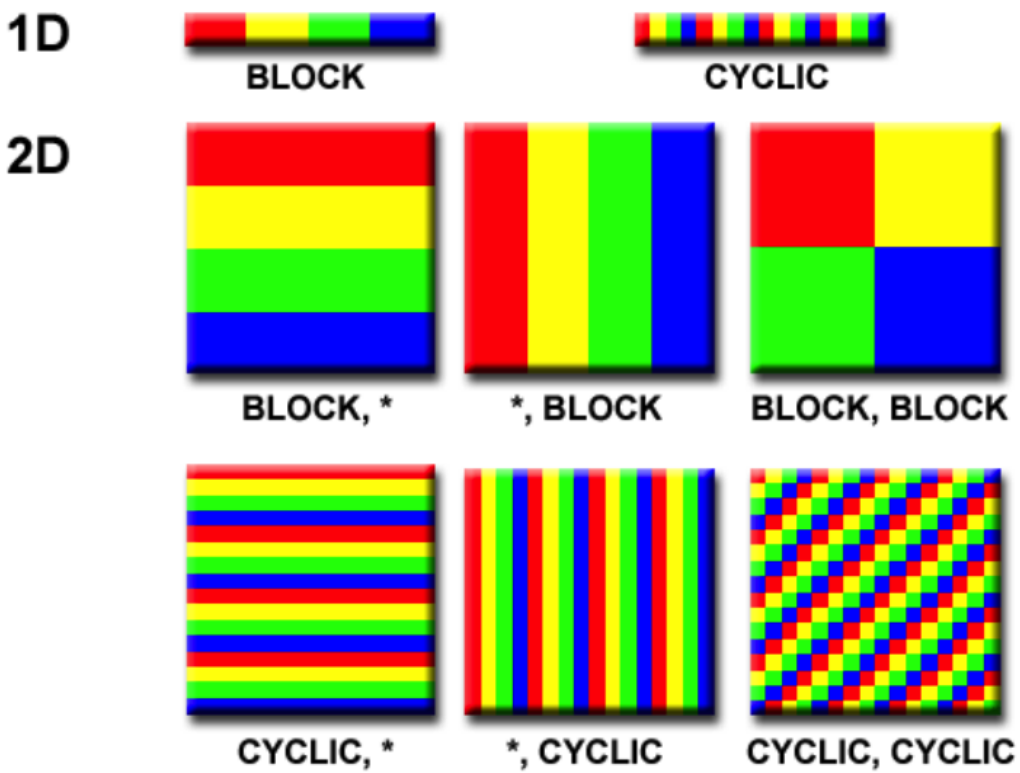
- Possible Matrix Partitioning



Distribute the workload to different threads

Part 1: matrix multiplication

- Other data partitioning strategies in matrix



Part 2: matrix multiplication comparison

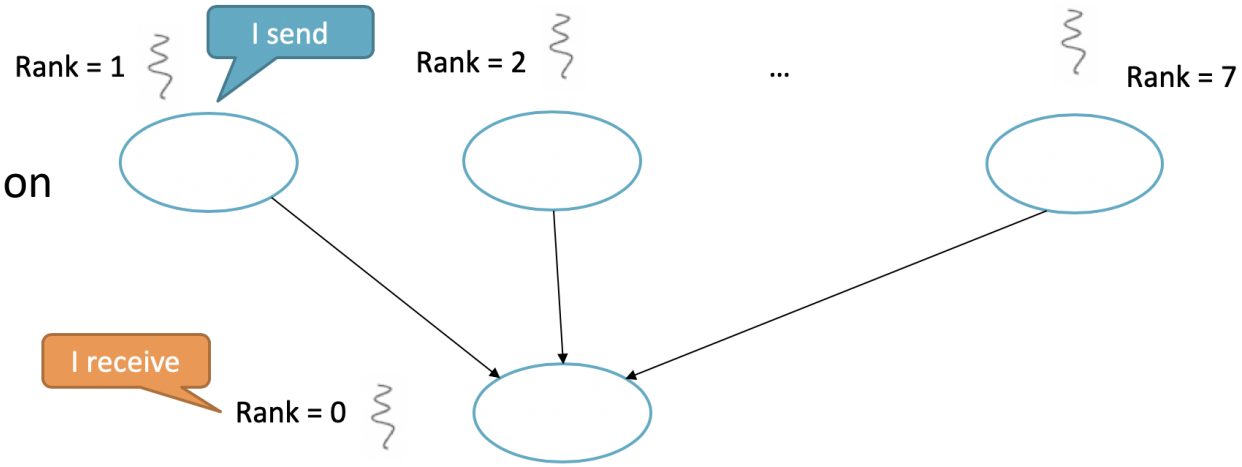
- Implement inter-process communication in matrix multiplication using
  - (a) MPI point-to-point communication (e.g., MPI_Send/MPI_Receive)
  - (b) MPI collective communication (e.g., MPI_Reduce/MPI_Allreduce/MPI_Broadcast, etc.)

  and compared the performance of the two implementations.

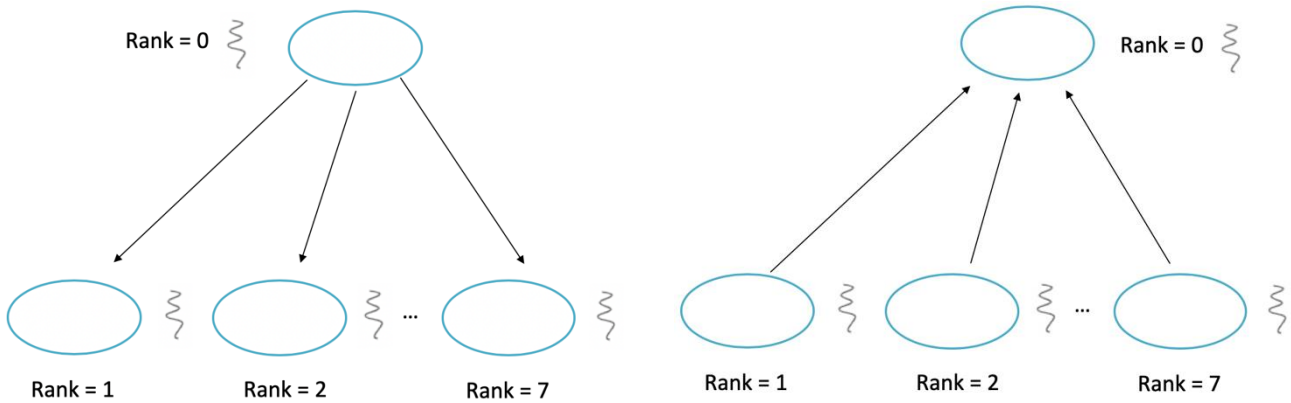
Order of Matrix	1024	2048	4096	8192
MPI point-to-point communication				
MPI collective communication				

Part 2: matrix multiplication comparison

Point-to-Point communication
MPI_Send/
MPI_Receive



MPI_Bcast/
MPI_Reduce/
MPI_Scatter/
MPI_Gather

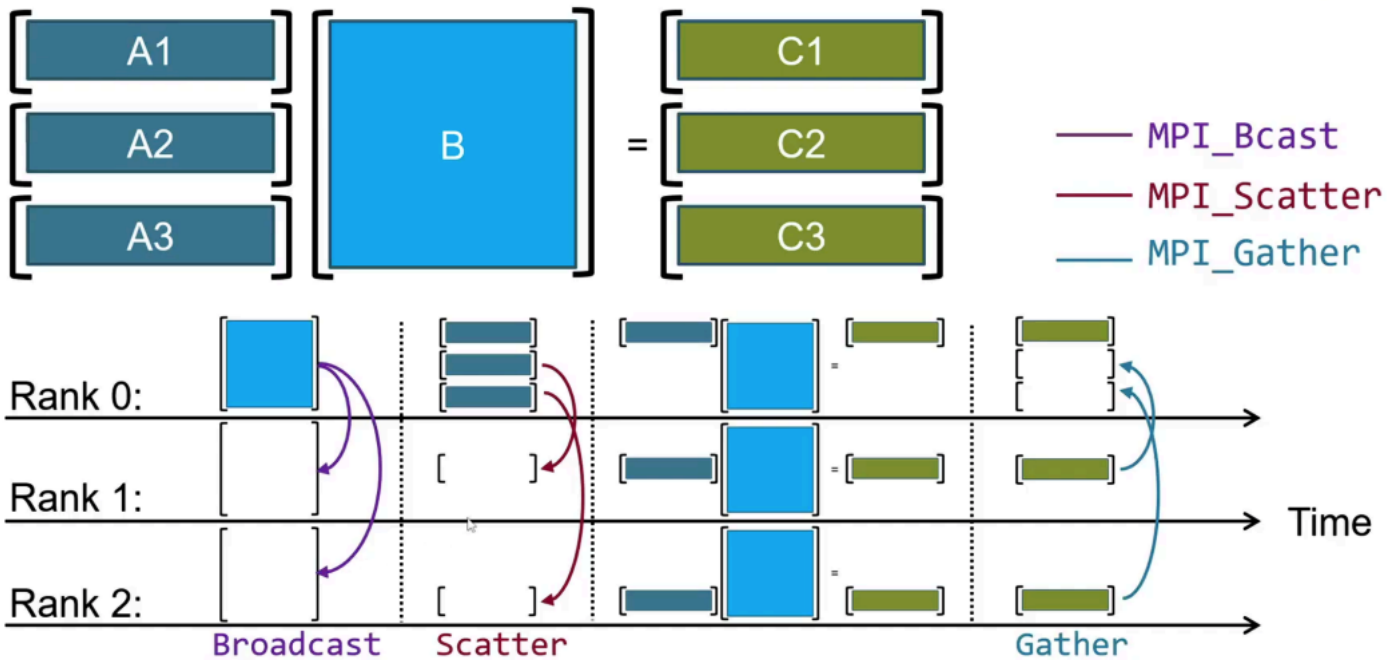


Part 2: matrix multiplication comparison

Data Movement	MPI Functions	Architectural Use Case
Direct 1-to-1	MPI_Send / MPI_Recv	Direct data handoffs. High flexibility, but inefficient for crowds.
Master to Workers	MPI_Scatter / MPI_Bcast	Scatter splits a large array into chunks; Bcast sends the exact same data to everyone.
Workers to Master	MPI_Gather / MPI_Reduce	Gather collects array chunks back together; Reduce aggregates data into a single value (sum, min, max).

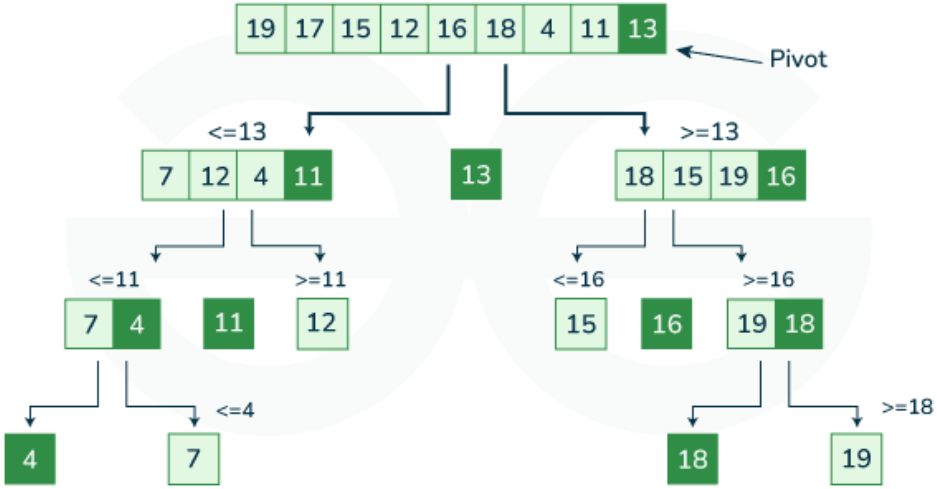
Part 2: matrix multiplication comparison

- GEMM Data Flow



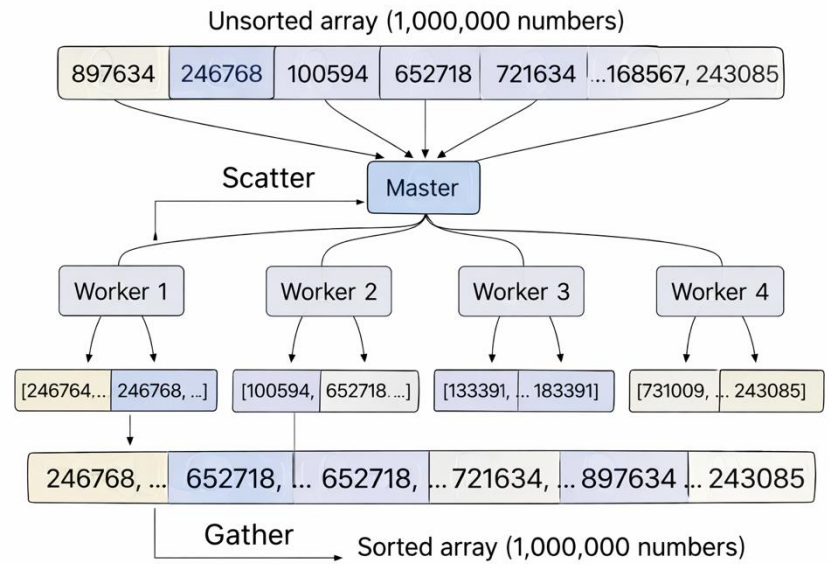
Part 3: Quicksort in MPI

- Divide and Conquer algorithm sorts the above 1 million unsorted numbers. However, the program executes in the sequential implementation. Please write a parallel program quick sort using MPI based on this sequential solution. Compare the parallel program execution time with the sequential version and write a report with data and figures introducing your implementation.



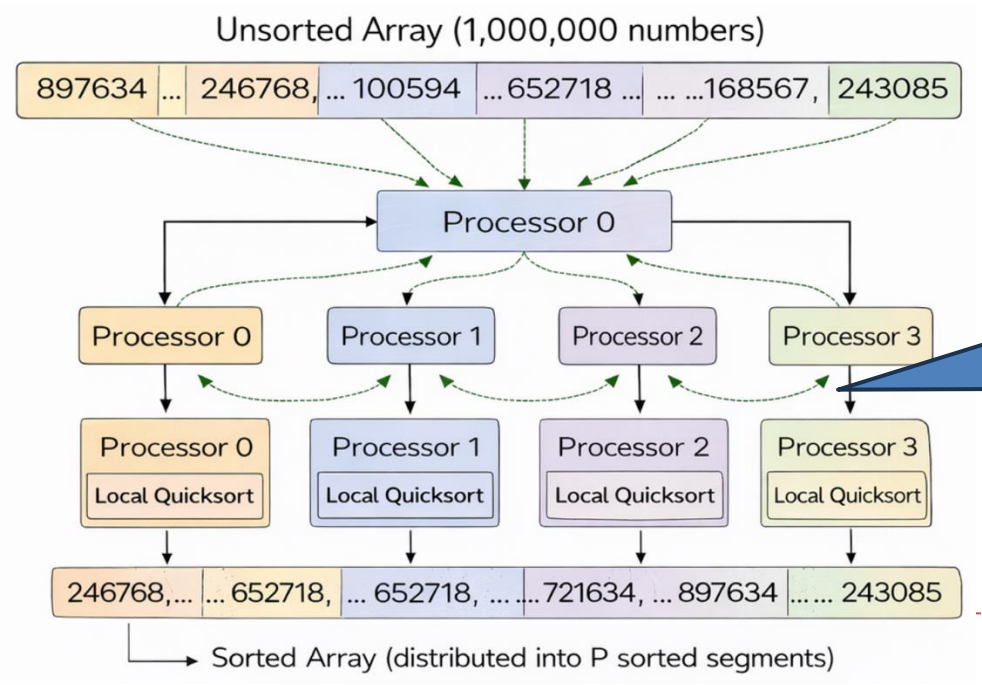
Part 3: Quicksort in MPI

- THE CONCEPT
  - QuickSort is inherently Divide and Conquer. Choose a pivot, partition the 1,000,000-element array into left/right sub-arrays, and sort recursively.
- THE CHALLENGE
  - Unlike GEMM's static grid, how do you map a recursive, dynamic tree structure onto a fixed number of static MPI ranks?

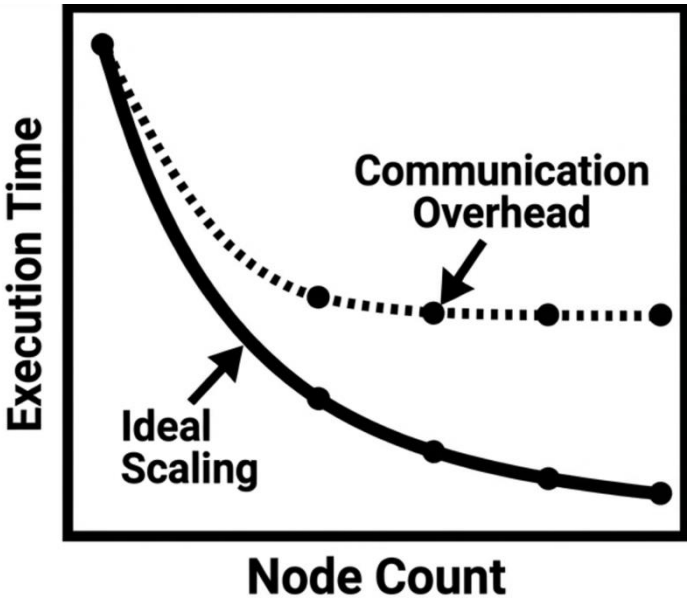


Part 3: Quicksort in MPI

- Instead of each process sorting independently and merging later, we can:
  - Repeatedly partitions data across processes using pivots
  - Each process ends up owning a portion of the final sorted range
  - Then each process runs a local quicksort



Core Idea: "Sorting by moving data to the right processor!

Part 3: Quicksort in MPI

- Timing
  - Use MPI_Wtime() to precisely measure parallel execution time (excluding verification).
- The Report
  - Document your architectural logic, provide code screenshots, and chart execution times (Sequential VS. Parallel).
- The Analysis
  - Demonstrate actual speedup. If parallel code is slower, analyze the communication overhead to explain why.